SENTINEL · ARCHITECTURE RETHINK

# You built a governed pipeline. Citi is asking for a governed model.

A critique of the current design, a reshaped control flow, an off-the-shelf toolkit, and the screens worth building. Nothing here is built yet.



PREPARED 2026-07-17 AUDIENCE: ONE STATUS: PROPOSAL, NOT SPEC

THE VERDICT

# Your governance layer isn't governing the LLM. It's governing ~~scikit-learn~~.

The current build has an audit log, RBAC, PII redaction, a cost ledger, an eval gate, and a human approval gate. Every one of them fires on a **logistic regression**. The language model sits at the end of the pipeline writing prose about numbers that were already computed.

So the harness is real, but it is auditing sklearn. If you turned the LLM off entirely, every control would still pass. That is the tell.

**The delta.** Today the artifact proves you can build an ML pipeline with an audit log. It should prove you can put a language model between a data scientist and a bank's customer data without losing your license.

The question in the room is **"can you deploy LLMs to help data scientists."** The honest answer to how LLMs help data scientists is: they write the code. Databricks Assistant. Hex Magic. Snowflake Copilot.

Which means the artifact has to govern **generated code, before it executes, against customer data**. Everything else in this deck follows from that one move.

PROBLEM ONE

# Where the LLM sits is the whole design

NOW — THE MODEL IS A COMMENTATOR, BOLTED TO THE END

| 01 | 02 | 03 | 04 | 05 | 06 | LLM |
|---|---|---|---|---|---|---|
| Load data | Profile | EDA | Fit model | Human gate | Validate | Narrates it |

Turn the last box off and nothing breaks. The controls never touch the model, because the model never touches anything.

PROPOSED — THE MODEL IS UPSTREAM OF EXECUTION, WITH GATES ON BOTH SIDES

| 01 | GATE | LLM | GATE | 04 | GATE | 06 |
|---|---|---|---|---|---|---|
| Ask | Policy + purpose | Writes the code | Static analysis | Sandbox exec | Disclosure | Attest |

Now the model is load-bearing, sitting between an analyst and real customer data. Every control has something dangerous to actually control.

PROBLEM TWO

# The platform story has nowhere to live

Half your stated goal is agent registry, templates, and scaffolding. A linear pipeline demonstrates none of it.

WHAT REGISTRY / TEMPLATE / SCAFFOLDING ACTUALLY CLAIMS

That the second and tenth analysis are cheaper than the first. That a central team knows what is deployed, who owns it, and when it was last evaluated. Neither claim is visible in a pipeline diagram.

THE VERSION THAT PROVES IT

Scaffolding is the **only** path to creating an agent, so an ungoverned agent is structurally impossible. The generated agent inherits the harness rather than re-implementing it.

THE LIFECYCLE THAT MAKES IT REAL


An agent cannot reach `certified` without passing its eval suite, carrying a named owner, and holding a model-risk signoff from someone who is not its author.

**Demo an agent failing certification.** Everyone shows the happy path. Showing the registry refuse to promote something, with reasons, is the platform PM proof, and almost nobody does it.

THE IDEA TO BUILD AROUND

# Four layers of leash

Don't pick one autonomy level and defend it. Ship all four, and let policy decide which one this person gets on this data right now.

| | | |
|---|---|---|
| **L0** | **It explains** | The model reads finished numbers and writes prose about them. It cannot touch data. `Always available. This is all you have today.` |
| **L1** | **It chooses** | The model picks a pre-approved analysis from the registry and fills in typed parameters. It writes no code. `Safe on the most sensitive data.` |
| **L2** | **It writes, inside a fence** | The model writes real code, but only against an allowlisted API. Checked before it runs. `The sweet spot. Build this one.` |
| **L3** | **It improvises** | The model writes near-arbitrary code in a sandbox with no network and no writes. `Synthetic and public data only.` |

`who they are × how sensitive the data is → which layer they get`

JUNIOR ON CUSTOMER PII

L1 ONLY

Pick from the catalog. No generated code goes anywhere near real accounts.

CERTIFIED ANALYST ON INTERNAL DATA

UP TO L2

Write code against the fenced API. Gate reviews it before execution.

ANYONE ON SYNTHETIC DATA

UP TO L3

Improvise freely. There is nothing here worth stealing.

**Why this is the centerpiece:** it is a product decision, not an engineering one. It is defensible to a regulator. And it makes the governance layer visibly *decide something* instead of merely writing to a log.

THE RESHAPED CONTROL FLOW

# Nine stages, and the control that fires at each

| | | |
|---|---|---|
| 01 | **Ask** | Bind identity and `purpose`. Resolve the tier. |
| 02 | **Plan** | Model may only select `certified` registry entries. |
| 03 | **Access** | RBAC, purpose limit, row and column policy. |
| 04 | **Generate** | The model writes code. Nothing runs yet. |
| 05 | **Gate** | Static analysis. Allowlist, no egress, no writes. |
| 06 | **Execute** | Sandbox. Off-the-shelf libraries do the math. |
| 07 | **Screen** | Disclosure control. Small cells, PII, leakage. |
| 08 | **Interpret** | Grounded narration. Faithfulness eval. |
| 09 | **Attest** | Evidence pack. Lineage. Signoff. |

LEGEND

`BRASS = DOES NOT EXIST TODAY`

Four of the nine are new, and they are the four that carry the entire argument. `Generate → Gate → Execute → Screen` is the span that proves the claim.

THEN: PUBLISH, TWO WAYS

**For the data scientist:** a reproducible notebook where the code is visible and diffable.

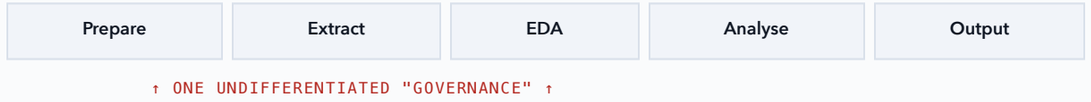
**For leadership:** the claim, its provenance, the controls that fired, and what the data does *not* let you say.

> Drop the phrase **"wrapped around by governance."** A perimeter is the wrong model and a sharp interviewer reads it as inexperience.

A FRAMING CORRECTION

# Controls are inline and differentiated, not a box you draw around the outside

WHAT YOU DREW




One box says every stage gets the same treatment. It doesn't. The control that matters at extraction has nothing in common with the control that matters at output.

WHAT A BANK ACTUALLY RUNS


RBAC and purpose bind at **Access**. Code safety at **Gate**. Small-cell suppression at **Screen**. SR 11-7 at **Attest**. Naming them separately is what signals you have done this before.

GAPS

# Six things your structure is missing entirely

01 **Purpose limitation**

"You may query customer data for fraud modelling, not for marketing." Bank-real, has no analogue in generic AI governance demos, and cheap to implement. The same query is legal or illegal depending on why you are asking.

02 **Output disclosure control**

If a cohort cell returns n=3, you cannot display it. Small-cell suppression and k-anonymity are real controls in every regulated context, and I have never seen anyone demo them in an AI governance artifact. Highest signal per line of code on this list.

03 **Fair lending, called by its real name**

Your fairness check is currently an ethics exercise. In a US bank it is ECOA and Reg B, with named protected classes and a specific legal test: disparate treatment versus disparate impact. Using the regulatory vocabulary is a large credibility delta for Citi specifically.

04 **Segregation of duties** CONFIRMED BUG

SR 11-7 *mandates* independent validation. Your `approve()` checks `actor.can_approve`, which is a *role* check, not an identity check. `RunState` never stores who started the run, so it cannot compare. And `mrm_approver` holds both `can_run` and `can_approve`. The same persona can approve its own run. The docstring calls this "the segregation-of-duties control." It isn't one yet.

05 **A POV on whether the LLM is a "model"**

Live, unsettled, and exactly what you will be asked. See the next slide. This is not trivia. The answer determines where the human gate physically sits.

06 **Lineage**

"Where did this number come from" is not optional in a bank. Every figure in the leadership report traces back to analysis, code, query, dataset version, and the policy decisions that applied along the way.

\+ **And one reversal: fairlearn**

You ruled it out to hand-roll fairness metrics "for auditability." Under the new thesis that inverts. If the pitch is "I govern off-the-shelf tools," hand-rolling the one metric a regulator cares most about undercuts it. **Agreed and confirmed.**

THE POV WORTH HAVING

# Is the LLM a "model" under SR 11-7?

You will get asked. The industry has not settled it. Having a crisp answer is most of the interview.

TOOL

## A human reviews the code.

The model proposes; a qualified person reads what it wrote and decides to run it. The model is an accelerant, like an IDE autocomplete. It inherits **no** validation burden, because a competent human is accountable for the output.

MODEL

## It autonomously produces a number that drives a decision.

Nobody reads the intermediate step. The output moves money, approves credit, or sizes a reserve. It inherits the **full** SR 11-7 burden: validation, monitoring, documentation, independent review.

**Why this is architecture, not philosophy.** The distinction tells you exactly where the human gate has to sit: at the last point where a person still meaningfully reviews before a number becomes a decision. Move the gate downstream of that line and you have silently reclassified your tool as a model, and taken on a validation obligation you are not meeting.

WHICH IS WHY THE LADDER AND THE GATE ARE THE SAME DESIGN

`L0/L1/L2` keep a human between generation and consequence, so they stay tools. `L3` without review would cross the line, which is precisely why L3 is fenced to synthetic data where no decision is downstream. The tiering is not a convenience feature. It is how you stay on the correct side of the definition.

THE TOOLKIT

# Buy the maths. Build the governance.

Nothing on this list is written by you. The claim is not "I can implement a clustering algorithm." The claim is "I can put a bank's controls around tools a data science team already uses."

| TOOL | STAGE | WHAT IT ACTUALLY DOES | WHY THIS ONE |
|---|---|---|---|
| **OPA / Rego** | ACCESS | A policy engine. You write rules in a dedicated language, and ask it "may this person do this thing" at runtime. | Policy becomes versioned, testable code with its own test suite, not `if role ==` buried in Python. This is the enterprise answer and it is recognised on sight. |
| **sqlglot** | ACCESS | Parses SQL into a syntax tree you can inspect and rewrite, then prints it back out. | Lets you reject restricted columns and *inject row filters* before a query ever executes. This is literally what a bank data platform does internally. |
| **DuckDB** | EXECUTE | A fast analytical database that runs in-process. Real SQL, no server. | Gives you a genuine query engine to enforce policy against, with no infrastructure. |
| **Presidio** | SCREEN | Detects and redacts personal data in text: names, accounts, national IDs. | Microsoft's, off-the-shelf, and vastly more defensible than the regex you have now. |
| **ydata-profiling** | EDA | One function call produces a full statistical profile of a dataframe as an HTML report. | Your entire exploratory stage, free, and a perfect thing to govern rather than write. |
| **Great Expectations** | ACCESS | Declares assertions about data ("this column is never null, and is between 0 and 1") and fails loudly when they break. | Already the data-quality vocabulary inside banks. Your data contracts become GE suites. |
| **statsmodels** | ANALYSE | Classical statistics. T-tests, proportion tests, regression with real inference output. | Boring and correct. Your A/B testing stage in one import. |
| **DoWhy / EconML** | ANALYSE | Causal inference. Forces you to state assumptions as a graph, then tests whether the effect survives them. | Microsoft's. Signals genuine sophistication, and the assumption-checking maps beautifully onto a governance narrative. |
| **lifelines** | ANALYSE | Survival analysis. Retention and time-to-event curves. | Your cohort analysis stage, free. |
| **fairlearn** | ANALYSE | Fairness metrics and mitigation across protected groups. | **The reversal.** Governing it beats reimplementing it, now that the thesis is "I govern off-the-shelf tools." |
| **SHAP** | ATTEST | Explains which features drove a given prediction. | Non-negotiable for model risk. Regulators ask "why did it decline this applicant." |
| **Evidently** | MONITOR | Detects drift between training data and what the model sees in production. | Standard. Ongoing monitoring is an explicit SR 11-7 requirement. |
| **OpenLineage** | ATTEST | An open standard for emitting "this job read X and wrote Y" events, assembling a provenance graph. | Standards-based lineage beats a homegrown graph, and answers "where did this number come from" by construction. |

| TOOL | STAGE | WHAT IT ACTUALLY DOES | WHY THIS ONE |
| --- | --- | --- | --- |
| **marimo** | PUBLISH | A reactive Python notebook that is stored as a plain `.py` file. | Notebooks you can actually code-review and git-diff. Notebook governance is a real, unglamorous bank pain point, and this quietly solves it. |
| **Quarto** | PUBLISH | Renders parameterised documents to HTML and PDF from code plus prose. | Reproducible leadership reports. Same input, same PDF, every time. |

HAVE VS. BUILD

# Less demolition than it sounds

| KEEP AS-IS | 9 |
|---|---|
| harness/audit | |
| harness/rbac | |
| harness/cost | |
| harness/tracing | |
| harness/model_card | |
| harness/eval_gate | |
| datasets/ | |
| rag/ | |
| app.py | |

| REPOINT | 5 |
|---|---|
| orchestrator.py | |
| gateway/ | |
| harness/identity | |
| harness/pii | |
| ml/fairness | |
| platform/registry | |

| GENUINELY NEW | 5 |
|---|---|
| policy/ | |
| codegen/ | |
| sandbox/ | |
| disclosure/ | |
| lineage/ | |

### THE TWO REAL CHANGES

**Where the LLM sits.** Narration becomes code generation.

**What the pipeline is.** Linear ETL becomes a request lifecycle.

### WHAT SURVIVES UNTOUCHED

LangGraph stays. Its `interrupt()` is still exactly the right human gate, and it is now sitting at the SR 11-7 tool-versus-model line, which makes it more defensible than before.

### IF YOU BUILD ONE THING

Build `Generate → Gate → Execute → Screen` at **L2**. That span alone proves the claim. Registry and the leadership view are half-built already and can be dragged forward.

**Honest scope warning:** this is bigger than what you have. The good news is the harness you already wrote is the part that usually takes longest, and almost all of it survives the reframe intact.

SCREENS · THE ANALYST

# What a data scientist sees

SENTINEL · CONSOLE L2

ASK

*"Does the credit model decline older applicants more often, holding income constant?"*

BINDING

SANDIP.DEV | QUANT ANALYST · CERTIFIED

GERMAN_CREDIT | INTERNAL

PURPOSE FAIR_LENDING_REVIEW

RESOLVED AUTONOMY

**L2** certified × internal → may write fenced code

Generate analysis | Browse catalog instead

**The point of this screen:** the tier is computed and displayed *before* anything happens. The analyst sees their leash, and so does the audit log.

GATE · PRE-EXECUTION REVIEW BLOCKED

GENERATED CODE — NOT YET EXECUTED

```
# fair lending: age vs. decline rate
import fairlearn.metrics as flm
df = ctx.table("german_credit")
m  = flm.MetricFrame(
      metrics=flm.selection_rate,
      y_true=df.y, y_pred=df.pred,
      sensitive_features=df.age_band)
requests.post(WEBHOOK, json=m.to_dict())
```

STATIC ANALYSIS

| | | |
|---|---|---|
| ✓ | Imports within allowlist | PASS |
| ✓ | No filesystem writes | PASS |
| ✓ | No `eval` / `exec` / dynamic import | PASS |
| ✓ | Columns permitted for purpose | PASS |
| × | Network egress on line 8 | BLOCKED |

**CTL-EGRESS-01.** Code attempted to send results to an external endpoint. Nothing was executed. Regenerate without the network call, or escalate to your data owner.

Regenerate | View policy

**The money screen.** A language model tried to exfiltrate results from a bank and the platform caught it statically, before execution, with a named control. This is the one to demo.

SCREENS · OVERSIGHT

# What everyone else sees

SCREEN · DISCLOSURE — 1 SUPPRESSED

DECLINE RATE BY AGE BAND

| BAND | N | RATE |
|---|---|---|
| 18–25 | 142 | 0.31 |
| 26–40 | 408 | 0.24 |
| 41–60 | 331 | 0.27 |
| 61–75 | 106 | 0.44 |
| *76+* | *3* | *suppressed* |

**CTL-DISC-02.** Cell n=3 is below the minimum of 10. Withheld from all downstream surfaces, including the model's own narration.

The number never reaches the LLM either. You cannot narrate what you were never shown.

REGISTRY — 6 AGENTS

CERTIFICATION STATUS

| | | |
|---|---|---|
| ✓ | `profiler` · v2.1 | CERTIFIED |
| ✓ | `fair-lending` · v1.4 | CERTIFIED |
| ✓ | `ab-test` · v1.0 | CERTIFIED |
| × | `cohort-retention` · v0.3 | REFUSED |

**Promotion refused.**

× faithfulness 0.72, floor is 0.90
× no independent validator (author = owner)
✓ owner assigned · ✓ data contract declared

Assign validator

Segregation of duties enforced by the registry, not by a policy memo nobody reads.

REPORT · LEADERSHIP — ATTESTED

FINDING

Applicants aged 61–75 are declined at **1.8×** the rate of the 26–40 band, and the gap holds after controlling for income and loan amount.

95% CI [1.31, 2.44] · n=1,000 · seed 42

WHERE THIS NUMBER CAME FROM

| | |
|---|---|
| ANALYSIS | `fair-lending v1.4` |
| DATASET | `german_credit@sha:4f2a1c` |
| QUERY | `q_88fe12 (row filter applied)` |
| CODE | `gen_5c1d90 (gate: passed)` |
| APPROVED | `r.mehta · Model Risk` |

CONTROLS ATTESTED

RBAC · PURPOSE · EGRESS · DISCLOSURE · FAITHFULNESS 1.0

**What this does not say.**

This is disparate *impact*, not intent. It is not a Reg B finding until Legal reviews business necessity. Age band 76+ was suppressed.

**The differentiator.** A dashboard is Tableau. A claim, its evidence chain, and an explicit statement of what you are *not* allowed to conclude is what a bank actually wants.

**Next.** Nothing here is built. If the reframe lands, the spec goes to `docs/features/` and the first slice is `Generate → Gate → Execute → Screen` at L2, against `german_credit`, with fairlearn doing the maths.